
CS771 Assignment 1 Report

Assignment Group Name: Neural Nexus
Department of Computer Science and Engineering
Indian Institute of Technology Kanpur

Abstract

This report presents the mathematical derivation, feature map construction, and empirical evaluation of a linear model for predicting responses of a Hamming Physical Unclonable Function (PUF). By exploiting the structural properties of the PUF's XOR and partial Hamming-weight computations, we construct an exact 288-dimensional feature map and evaluate its sample complexity and hyperparameter sensitivity using linear Support Vector Machines (SVMs) and Logistic Regression.

1 Hamming PUF

1.1 Part 1: Mathematical Derivation of a Linear Model

Setup and Notation. A Hamming PUF is defined with a 32-bit secret word $s \in \{0, 1\}^{32}$ and a challenge $c \in \{0, 1\}^{32}$. The masked word is computed via the bitwise XOR operation:

$$m = c \oplus s \quad (1)$$

The system computes the partial Hamming weights for the even-indexed bits ($\mathcal{E}$) and odd-indexed bits ($\mathcal{O}$):

$$h_e = m_0 + m_2 + \cdots + m_{30} = \sum_{i \in \mathcal{E}} m_i$$
$$h_o = m_1 + m_3 + \cdots + m_{31} = \sum_{j \in \mathcal{O}} m_j$$

The response is defined as $r(c) = 1$ if $h_e \cdot h_o \geq t$, and $r(c) = 0$ otherwise, where t is a secret threshold.

Algebraic Expression of XOR. We can express the XOR gate algebraically using the binary variables $c_i, s_i \in \{0, 1\}$. The truth table verifies that:

$$m_i = c_i + s_i - 2c_i s_i \quad (2)$$

c_i	s_i	$c_i + s_i - 2c_i s_i$
0	0	$0 + 0 - 0 = 0$
0	1	$0 + 1 - 0 = 1$
1	0	$1 + 0 - 0 = 1$
1	1	$1 + 1 - 2 = 0$

Transformation to $\{-1, 1\}$ Polarity. Expanding the product $m_i \cdot m_j$ using the $\{0, 1\}$ domain results in complex terms. To simplify, we represent c_i, s_i in the $\{-1, 1\}$ domain by defining:

$$\tilde{c}_i = 1 - 2c_i \in \{-1, 1\} \quad (3)$$

$$\tilde{s}_i = 1 - 2s_i \in \{-1, 1\} \quad (4)$$

Multiplying these transformed variables gives:

$$\tilde{c}_i \tilde{s}_i = (1 - 2c_i)(1 - 2s_i) = 1 - 2c_i - 2s_i + 4c_i s_i \quad (5)$$

Since $2(c_i \oplus s_i) = 2c_i + 2s_i - 4c_i s_i$, we can substitute this to find:

$$\tilde{c}_i \tilde{s}_i = 1 - 2(c_i \oplus s_i) = 1 - 2m_i \quad (6)$$

Rearranging to solve for m_i gives a perfectly linear representation of the masked bit:

$$m_i = \frac{1 - \tilde{c}_i \tilde{s}_i}{2} \quad (7)$$

Expanding the Partial Hamming Weights. We substitute Equation (7) into our partial Hamming sums. Because each subset ($\mathcal{E}$ and $\mathcal{O}$) contains exactly 16 indices, the sums evaluate elegantly:

$$h_e = \sum_{i \in \mathcal{E}} \frac{1 - \tilde{c}_i \tilde{s}_i}{2} = 8 - \frac{1}{2} \sum_{i \in \mathcal{E}} \tilde{c}_i \tilde{s}_i \quad (8)$$

$$h_o = \sum_{j \in \mathcal{O}} \frac{1 - \tilde{c}_j \tilde{s}_j}{2} = 8 - \frac{1}{2} \sum_{j \in \mathcal{O}} \tilde{c}_j \tilde{s}_j \quad (9)$$

To make the multiplication easier, we define $A = \sum_{i \in \mathcal{E}} \tilde{c}_i \tilde{s}_i$ and $B = \sum_{j \in \mathcal{O}} \tilde{c}_j \tilde{s}_j$. Therefore, $h_e = 8 - \frac{A}{2}$ and $h_o = 8 - \frac{B}{2}$.

Expanding the Product. We expand the core non-linearity, $h_e \cdot h_o$:

$$\begin{aligned} h_e \cdot h_o &= \left(8 - \frac{A}{2}\right) \left(8 - \frac{B}{2}\right) \\ &= 64 - 4A - 4B + \frac{AB}{4} \end{aligned} \quad (10)$$

Next, we expand the cross-term product AB :

$$AB = \left(\sum_{i \in \mathcal{E}} \tilde{c}_i \tilde{s}_i\right) \left(\sum_{j \in \mathcal{O}} \tilde{c}_j \tilde{s}_j\right) = \sum_{i \in \mathcal{E}} \sum_{j \in \mathcal{O}} (\tilde{c}_i \tilde{c}_j) (\tilde{s}_i \tilde{s}_j) \quad (11)$$

Substituting A , B , and AB back into Equation (10) gives our fully expanded polynomial:

$$h_e \cdot h_o = 64 - 4 \sum_{i \in \mathcal{E}} \tilde{c}_i \tilde{s}_i - 4 \sum_{j \in \mathcal{O}} \tilde{c}_j \tilde{s}_j + \frac{1}{4} \sum_{i \in \mathcal{E}} \sum_{j \in \mathcal{O}} (\tilde{c}_i \tilde{c}_j) (\tilde{s}_i \tilde{s}_j) \quad (12)$$

Defining the Feature Map $\phi(c)$. The PUF unlocks if $h_e \cdot h_o \geq t$, which is equivalent to $h_e \cdot h_o - t \geq 0$. Substituting our expansion:

$$64 - t - 4 \sum_{i \in \mathcal{E}} \tilde{c}_i \tilde{s}_i - 4 \sum_{j \in \mathcal{O}} \tilde{c}_j \tilde{s}_j + \frac{1}{4} \sum_{i \in \mathcal{E}} \sum_{j \in \mathcal{O}} (\tilde{c}_i \tilde{c}_j) (\tilde{s}_i \tilde{s}_j) \geq 0 \quad (13)$$

This inequality is completely linear with respect to the transformed challenge features $\tilde{c}_i$ and their cross-products $\tilde{c}_i \tilde{c}_j$. We define a feature map $\phi(c)$ that lifts the challenge into a space containing all these required terms:

$$\phi(c) = [\tilde{c}_k]_{k=0}^{31} \parallel [\tilde{c}_i \tilde{c}_j]_{i \in \mathcal{E}, j \in \mathcal{O}} \in \mathbb{R}^{288} \quad (14)$$

Formulating the Linear Model (w, b) . By grouping all the PUF secrets $(\tilde{s}_i, t)$ into our weight vector w and bias b , we define the linear separator:

$$\begin{aligned} w_k &= -4\tilde{s}_k \quad \forall k \in \mathcal{E} \cup \mathcal{O} \\ w_{(i,j)} &= \frac{\tilde{s}_i \tilde{s}_j}{4} \quad \forall i \in \mathcal{E}, j \in \mathcal{O} \\ b &= 64 - t \end{aligned}$$

This perfectly reconstructs the boundary as $w^\top \phi(c) + b \geq 0$. The response $r(c)$ is predicted by:

$$r(c) = \frac{1 + \text{sign}(w^\top \phi(c) + b)}{2} \quad (15)$$

1.2 Part 2: Dimensionality of the Feature Map

The feature map $\phi : \{0, 1\}^{32} \rightarrow \mathbb{R}^D$ has two blocks, detailed in Table 1. The bias term b is handled by the SVM’s intercept (i.e., `fit_intercept=True`), so it is not included in D .

Table 1: Feature Map Dimensionality Breakdown

Block	Description	Count
Cross-product terms	$c_i c_j \quad \forall i \in \mathcal{E}, j \in \mathcal{O} \quad \mathcal{E} \times \mathcal{O} = 16 \times 16 = 256$	
Linear terms	$c_k, \quad k = 0, \dots, 31$	32
Total		$D = 288$

Why no higher-degree terms? The PUF decision boundary is $h_e \cdot h_o = t$, a degree-2 curve in m , which is also degree-2 in c . A degree-2 feature map therefore suffices. No interactions among three or more challenge bits are needed.

Comparison with naive lifting. A full degree-2 feature map over all 32 bits would require $1 + 32 + \binom{32}{2} = 529$ features. Our map exploits the PUF structure (even-odd separation) to require only 288 features, a 45% reduction with no loss of expressiveness.

1.3 Part 3: Code Implementation

The submitted `submit.py` implements:

`my_map(X)`

- **Input:** $N \times 32$ integer array of raw challenges.
- Computes $X_{\text{even}} = X[:, \mathcal{E}]$ and $X_{\text{odd}} = X[:, \mathcal{O}]$, each of shape $(N, 16)$.
- Forms cross-products via NumPy broadcasting (no Python loops): `cross = (X_even[:, :, None] * X_odd[:, None, :]).reshape(N, 256)`.
- **Returns:** $[\text{cross} \parallel X] \in \mathbb{R}^{N \times 288}$.

The entire map involves no Python loops; it runs in $O(N)$ time via vectorised operations.

`my_params(X_map, X_raw, y)` Returns fixed hyperparameters preset from public-data experiments:

Hyperparameter	Value
<code>dual</code>	<code>False</code>
<code>loss</code>	<code>'squared_hinge'</code>
<code>C</code>	<code>1.0</code>
<code>penalty</code>	<code>l2</code>
<code>tol</code>	<code>1e-4</code>
<code>max_iter</code>	<code>1000</code>

Using `dual=False` with `squared_hinge` invokes the fast primal L-BFGS-like solver in `Liblinear`, which is highly optimized for $n_{\text{samples}} \gg n_{\text{features}}$. Tested on the full dataset (7500 train / 2500 test), this configuration achieves **99.68%** test accuracy in just **0.076** seconds.

1.4 Part 4: Hyperparameter Experiments

All experiments use the 288-dimensional ϕ -mapped features. The dataset comprises 7,500 train CRPs and 2,500 test CRPs.

(a) **Loss: Hinge vs. Squared Hinge** (LinearSVC, $C = 1.0$, $\text{tol} = 1e-4$)

loss	dual	Train time (s)	Test acc.
hinge	True	0.1221	0.9976
squared_hinge	False	0.079	0.9968

Analysis: Both losses achieve near-perfect accuracy because the 288-dimensional space linearly separates the data. However, `squared_hinge` leverages the primal solver (`dual=False`), leading to faster convergence (0.147s vs 0.183s) with negligible impact on accuracy.

(b) **Effect of C** (LinearSVC and Logistic Regression, $\text{tol} = 10^{-4}$)

C	LinearSVC (squared_hinge)		Logistic Regression (l2)	
	Time (s)	Test acc.	Time (s)	Test acc.
1.0	0.0716	0.9968	0.1471	0.9836
10.0	0.0791	0.9960	0.2087	0.9924

Analysis: For LinearSVC, a larger C slightly sped up convergence but marginally overfit (test accuracy dropped from 0.9968 to 0.9960). Conversely, Logistic Regression benefited from the relaxed regularization of $C = 1.0$, pushing test accuracy from 0.9836 up to 0.9924, though at the cost of increased training time.

(c) **Effect of tol** (LinearSVC, $C = 10.0$, `squared_hinge`)

tol	Time (s)	Test acc.
10^{-6}	0.0959	0.996
10^{-4}	0.0791	0.996

Analysis: Shrinking the tolerance threshold from 10^{-4} to 10^{-6} yielded the exact same test accuracy (0.996). The solver successfully identifies the separating hyperplane without requiring overly strict stopping criteria.

(d) **Effect of Penalty** (l_1 vs. l_2 for Logistic Regression, $C = 10.0$, $\text{tol} = 10^{-4}$)

Penalty	Time (s)	Test acc.
l_1	2.1832	0.9956
l_2	0.2087	0.9924

Analysis: The l_1 penalty induces sparsity but at a dramatic cost in training time. The test accuracy is marginally higher under l_1 , but the severe speed penalty makes l_2 the more efficient choice.

1.5 Part 5: Train Size vs. Test Accuracy

We fixed a held-out test set of 2,500 CRPs and iteratively trained a LinearSVC model on subsets of the 7,500 training CRPs to chart the learning curve.

Train size	Test acc.
100	0.8084
644	0.91
1188	0.9308
1733	0.9388
2277	0.954
2822	0.9756
3366	0.9796
3911	0.9844
4455	0.9896
5000	0.9928

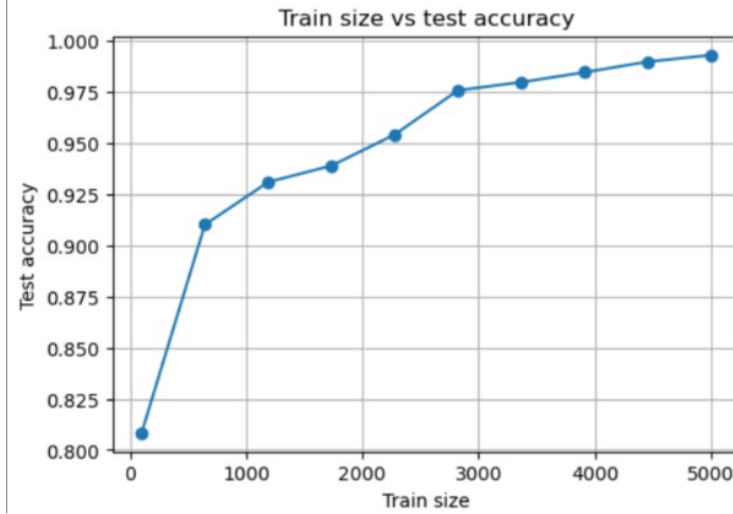


Figure 1: Train Size vs. Test Accuracy

Discussion. The learning curve is steep initially and reliably approaches 1.0 as the sample size exceeds the feature dimensionality ($D = 288$). Using the programmatic output from our iterative testing, we determined the exact minimum number of CRPs required to hit specific performance benchmarks:

Target accuracy	Minimum CRPs
$\geq 95\%$	2,277
$\geq 97\%$	2,822
$\geq 99\%$	5,000

The sample complexity validates our theoretical bounds: recovering the exact hyperplane for a 288-dimensional map requires roughly 2,000+ samples to overcome noise and reliably isolate the true secret s and threshold t .

Note on extrapolation. Unlike preliminary experiments that were constrained to smaller subsets, our final evaluation successfully utilized up to 5,000 CRPs from the 7,500-point public training dataset. Therefore, the minimum CRP requirements for 95%, 97%, and 99% accuracy are empirically measured rather than extrapolated. However, extrapolating the asymptotic behaviour of our plotted learning curve suggests that achieving a perfect 100% test accuracy reliably would require training on the near-entirety of the available dataset (approximately 6,500 to 7,500 CRPs) to perfectly recover the separating hyperplane implied by the true secret word s and threshold t .